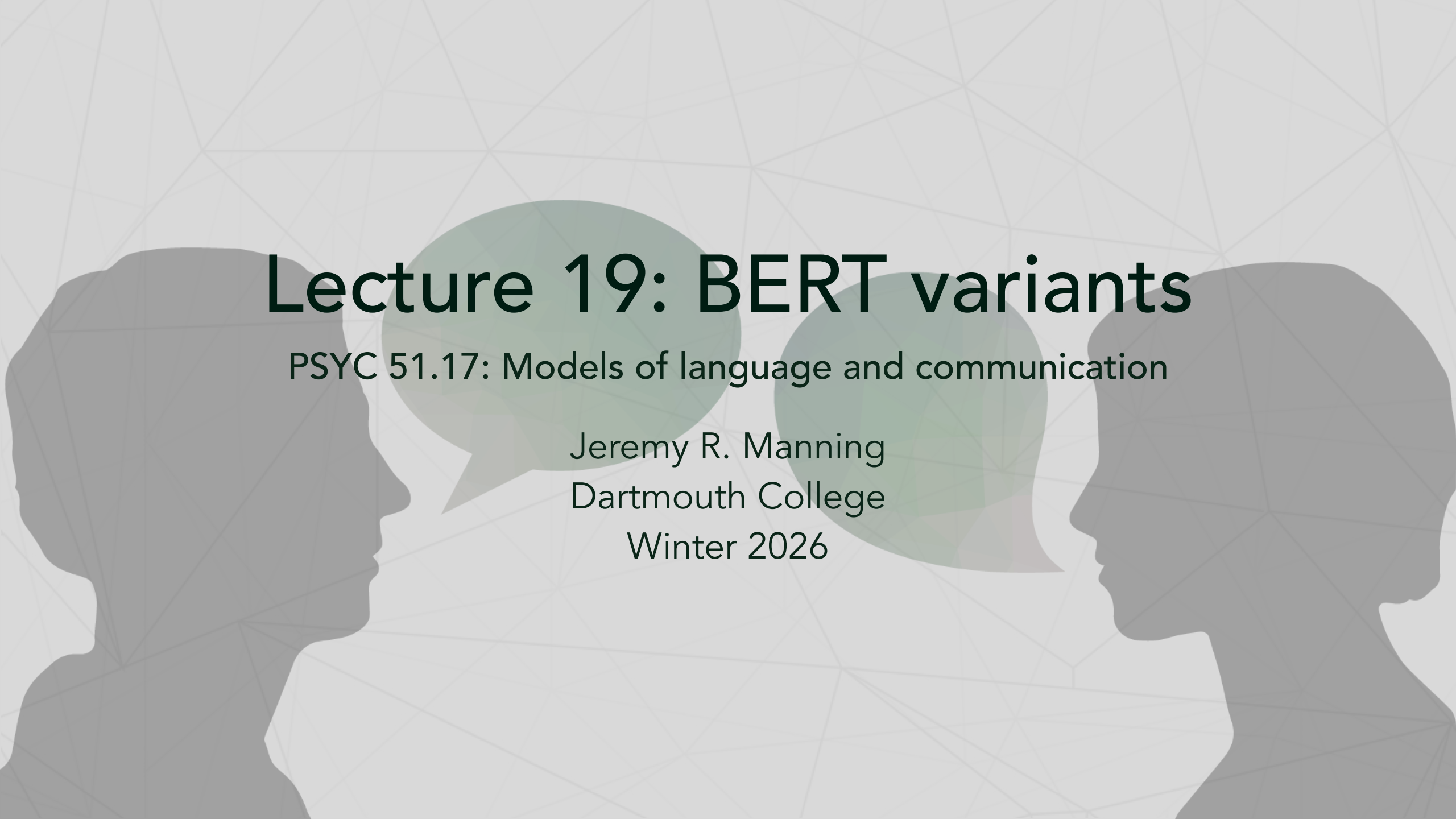


Lecture 19: BERT variants

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Identify the key limitations of the original BERT training procedure
2. Explain how RoBERTa, ALBERT, DistilBERT, and ELECTRA each address different BERT limitations
3. Compare parameter efficiency, training efficiency, and inference speed across variants
4. Select the appropriate BERT variant for a given task and constraint
5. Use HuggingFace to load and swap between different BERT variants

BERT's limitations

What could be improved?

Training procedure:

- Next Sentence Prediction (NSP) may not be useful
- Static masking — same masks are reused every epoch
- Some hyperparameter choices seemed arbitrary

Model size:

- 110M (Base) or 340M (Large) parameters
- Large memory footprint for deployment
- Slow inference for real-time applications

Training efficiency:

- Only 15% of tokens provide a training signal (the masked ones)
- 85% of computation is "wasted" on non-masked positions

Data and compute:

- Trained on only 3.3B words — modern datasets are much larger

RoBERTa: robustly optimized BERT

Key idea: better training = better performance

RoBERTa (Liu et al., 2019) keeps BERT's architecture but fixes the training recipe:

1. **Remove NSP** — Next Sentence Prediction hurt performance. Use only MLM with full sentences.
2. **Dynamic masking** — Generate a new masking pattern every time a sequence is seen, instead of reusing the same mask.
3. **Larger batches, more data** — Batch size 8K sequences (vs BERT's 256), 160GB text (vs 16GB), 500K steps (vs 100K).
4. **Longer sequences** — Train on longer contiguous text for better long-range understanding.

Takeaway

Training procedure matters as much as architecture. RoBERTa shows that BERT was significantly **undertrained**.

Dynamic vs static masking

How masking patterns are generated

Static masking (BERT):

- Mask tokens once during preprocessing
- Same masks reused every epoch
- Example: "My [MASK] is cute" → same every time
- Risk: model memorizes mask positions

Dynamic masking (RoBERTa):

- Generate new masks on-the-fly during training
- Different masks each time the same sequence is seen
- Epoch 1: "My [MASK] is cute"
- Epoch 2: "My dog [MASK] cute"
- Epoch 3: "My dog is [MASK]"
- Result: more diverse training signal, better generalization

RoBERTa results

Consistent improvements over BERT

Task	BERT-Large	RoBERTa	Improvement
SQuAD 2.0	83.1	89.4	+6.3
MNLI	86.7	90.2	+3.5
SST-2	94.9	96.4	+1.5
RACE	72.0	83.2	+11.2

Key findings

- Dynamic masking consistently outperforms static masking
- More data + longer training = substantial gains
- Removing NSP improves downstream task performance
- Some tasks see enormous gains (RACE: +11.2 points!)

ALBERT: a lite BERT

Key idea: parameter sharing for efficiency

ALBERT (Lan et al., 2019) dramatically reduces BERT's parameter count through two innovations:

1. Factorized embedding parameters:

- BERT: vocabulary (30K) × hidden size (768) = 23M parameters
- ALBERT: vocabulary (30K) × embedding size (128) + embedding (128) × hidden (768) = 3.9M parameters
- **83% fewer embedding parameters**

2. Cross-layer parameter sharing:

- All 12 transformer layers share the *same* weights
- Like running the same layer 12 times (iterative refinement)
- **89% fewer transformer parameters**

3. Sentence Order Prediction (SOP):

- Replaces NSP with a harder task: are sentences A, B in correct order or

ALBERT parameter efficiency

Dramatic parameter reduction

Model	Layers	Hidden size	Parameters
BERT-base	12	768	110M
ALBERT-base	12	768	**12M**
ALBERT-large	24	1024	18M
ALBERT-xlarge	24	2048	60M
ALBERT-xxlarge	12	4096	235M

Factorized embedding in code

```
import torch.nn as nn

# BERT: direct embedding (30K vocab × 768 hidden = 23M params)
bert_embed = nn.Embedding(30000, 768)
```

ALBERT parameter efficiency

Factorized embedding in code

```
9 # 83% fewer embedding parameters!
```

...continued

Cross-layer parameter sharing

BERT vs ALBERT layer structure

```
class BERT:
    def __init__(self):
        # Each layer has unique parameters
        self.layers = [TransformerLayer() for _ in range(12)]
        # 12 × 7M params = 85M params in transformer layers

    def forward(self, x):
        for layer in self.layers:
            x = layer(x)   # Different weights each time
        return x

class ALBERT:
    def __init__(self):
        # Single shared layer
        self.shared_layer = TransformerLayer()
```

continued...

Cross-layer parameter sharing

BERT vs ALBERT layer structure

```
            # 1 × 7M params = 7M params (89% reduction!)

        def forward(self, x):
            for _ in range(12):
                x = self.shared_layer(x)  # Same weights reused
            return x
```

...continued

Trade-off

ALBERT has 89% fewer parameters but the **same compute cost** — it still performs 12 forward passes through the transformer layer. The savings are in memory, not speed.

DistilBERT: knowledge distillation

Key idea: train a small model to mimic a large model

Knowledge distillation (Sanh et al., 2019) compresses BERT into a smaller, faster model:

- **Teacher**: Full BERT-base (12 layers, frozen)
- **Student**: DistilBERT (6 layers, trainable)
- **Training signal**: Student learns to match the teacher's *soft probability distributions*, not just the hard labels

The teacher's "wrong" predictions contain useful information — e.g., predicting "cat" is more likely than "car" for a masked animal slot tells the student about semantic similarity.

Results

Metric	BERT-base	DistilBERT	Change
Parameters	110M	66M	40% smaller
Inference speed	1×	1.6×	60% faster
GLUE score	79.6	77.0	97% retained

DistilBERT training

Knowledge distillation training loop

```
teacher = BertModel.from_pretrained("bert-base")  # 12 layers, frozen
student = DistilBertModel(num_layers=6)           # 6 layers, trainable

for batch in training_data:
    # Teacher provides "soft targets" (probability distributions)
    with torch.no_grad():
        teacher_logits = teacher(batch)  # e.g., [0.7, 0.2, 0.1, ...]

    # Student tries to match teacher's distribution
    student_logits = student(batch)

    # Distillation loss: KL divergence between soft
```

continued...

DistilBERT training

Knowledge distillation training loop

```
        loss_distill = KL_divergence(
            softmax(student_logits / T),
            softmax(teacher_logits / T)
        )
        loss_mlm = masked_lm_loss(student_logits, labels)

        loss = 0.5 * loss_distill + 0.5 * loss_mlm
```

...continued

ELECTRA: efficient learning from all tokens

Key idea: learn from 100% of tokens, not just 15%

ELECTRA (Clark et al., 2020) uses a **generator-discriminator** setup:

1. A small **generator** (like a mini-BERT) fills in masked tokens with plausible replacements
2. A **discriminator** classifies *every* token as original or replaced
3. The discriminator provides a training signal for **all tokens** — not just the 15% that were masked

ELECTRA training example

```
sentence  = "The chef cooked a delicious meal"
masked    = "The chef [MASK] a delicious meal"

# Small generator fills in the mask
generator_output = generator(masked)  # Predicts: "ate" (plausible but wrong)
corrupted = "The chef ate a delicious meal"
```

ELECTRA: efficient learning from all tokens

ELECTRA training example

```
discriminator(corrupted)
# Output: [orig, orig, REPLACED, orig, orig, orig]
# Loss computed on ALL 6 tokens (not just 1 masked token!)
```

...continued

ELECTRA efficiency

Learning from every token

BERT: Learns from 15% of tokens (masked ones only) — 85% of compute generates no training signal.

ELECTRA: Learns from 100% of tokens (all get a real/replaced label) — every position contributes to learning.

Result: ELECTRA reaches BERT-level performance with **4× less compute**.

When to use ELECTRA

ELECTRA is ideal when you have limited compute budget:

- ELECTRA-Small outperforms BERT-Small
- ELECTRA-Base is competitive with BERT-Large
- With the same compute budget, ELECTRA consistently wins

Variant comparison summary

Choosing the right BERT variant

Model	Key innovation	Best for
BERT	MLM + NSP	Baseline, well-understood
RoBERTa	Better training recipe	Maximum quality
ALBERT	Parameter sharing	Memory-constrained deployment
DistilBERT	Knowledge distillation	Speed-critical production
ELECTRA	Replaced token detection	Limited training budget

General guidelines:

- **Best quality:** RoBERTa-Large
- **Best efficiency:** DistilBERT
- **Limited memory:** ALBERT
- **Limited training budget:** ELECTRA
- **Good default:** RoBERTa-Base or BERT-Base

Other notable BERT variants

The BERT family keeps growing

DeBERTa (Microsoft, 2020): Disentangled attention separates content and position representations. Enhanced mask decoder. State-of-the-art on SuperGLUE.

SpanBERT (Facebook, 2019): Masks random contiguous *spans* instead of individual tokens. Span boundary objective. Better for extractive tasks (QA, coreference).

ERNIE (Baidu, 2019): Entity-level and phrase-level masking. Knowledge-enhanced pre-training. Strong on Chinese NLP tasks.

BART (Facebook, 2019): Encoder-decoder architecture (not encoder-only). Denoising autoencoder with various corruption strategies. Excellent for generation tasks.

Model selection guide

How to choose the right model for your task

Step 1: What are your constraints?

- Need maximum quality? → **RoBERTa-Large**
- Need fast inference? → **DistilBERT**
- Need small memory footprint? → **ALBERT**
- Limited training compute? → **ELECTRA**
- Need text generation? → Consider **BART** or **GPT** instead

Step 2: Consider your domain

- Biomedical text? → **BioBERT**, **PubMedBERT**
- Scientific text? → **SciBERT**
- Legal text? → **LegalBERT**
- Multilingual? → **mBERT**, **XLM-RoBERTa**

Step 3: Start simple, iterate

- Begin with `bert-base-uncased` as a baseline

Using different variants with HuggingFace

Easy model switching with AutoModel

```
from transformers import AutoModel, AutoTokenizer

# All variants share the same API – just change the model name!

# BERT
model = AutoModel.from_pretrained("bert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# RoBERTa
model = AutoModel.from_pretrained("roberta-base")
tokenizer = AutoTokenizer.from_pretrained("roberta-base")

# ALBERT
```

continued...

Using different variants with HuggingFace

Easy model switching with AutoModel

```
14  model = AutoModel.from_pretrained("albert-base-v2")
15  tokenizer = AutoTokenizer.from_pretrained("albert-base-v2")
16
17  # DistilBERT
18  model = AutoModel.from_pretrained("distilbert-base-uncased")
19  tokenizer = AutoTokenizer.from_pretrained("distilbert-base-
    uncased")
20
21  # ELECTRA
22  model = AutoModel.from_pretrained("google/electra-base-
    discriminator")
23  tokenizer = AutoTokenizer.from_pretrained("google/electra-base-
    discriminator")
```

...continued

Benchmarking variants

Comparing models on sentiment analysis

```
import time
from transformers import pipeline

models = {
    "bert-base": "textattack/bert-base-uncased-SST-2",
    "distilbert": "distilbert-base-uncased-finetuned-sst-2-english",
    "albert": "textattack/albert-base-v2-SST-2",
}
test_texts = ["This movie was fantastic!", "I hated every minute."] * 100

for name, model_id in models.items():
    pipe = pipeline("sentiment-analysis", model=model_id)
    start = time.time()
    results = pipe(test_texts)
```


Discussion

Questions to consider

1. **Training vs architecture:** RoBERTa shows that training matters enormously. Is architecture innovation overrated? How much can we improve just by training longer and better?
2. **Parameter efficiency:** ALBERT shares all layers and still works well. Why? What does this tell us about what different layers learn? Is there a "sweet spot" for sharing?
3. **Knowledge distillation:** Why does the student learn *better* from the teacher's soft probabilities than from hard labels? What information is encoded in the teacher's distribution over wrong answers?
4. **Model selection in practice:** How do you decide which variant to use for a real project? Is it worth fine-tuning multiple variants and

References

Further reading

Liu et al. (2019, *arXiv*) "RoBERTa: A Robustly Optimized BERT Pretraining Approach"

Lan et al. (2019, *ICLR*) "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations"

Sanh et al. (2019, *NeurIPS Workshop*) "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter"

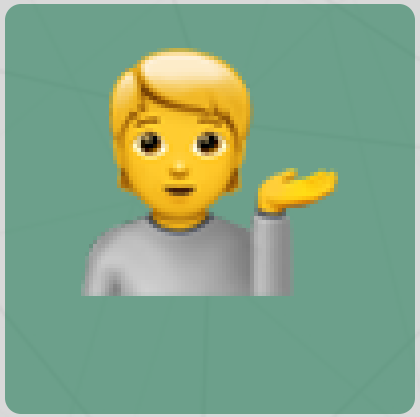
Clark et al. (2020, *ICLR*) "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators"

He et al. (2020, *ICLR*) "DeBERTa: Decoding-enhanced BERT with Disentangled Attention"

Questions?

Email

Discord



Office hours

Up next...

Applications of encoder models: from Google Search to cognitive neuroscience